Slices, Maps, Strings & Formatting Cheatsheet

Lessons: 07 — Arrays, Slices & Maps · 08 — Strings, Runes, Bytes & Formatting

Examples: 07 · 08

Covers: builtins, `slices`, `maps`, `strings`, `strconv`, `bytes`, `unicode/utf8`, `fmt`

Legend: `[*]` = real Go API that the lessons have not covered yet

ARRAYS (fixed length, a value)

```
var a [3]int                 zero-valued, length is part of the TYPE
a := [3]int{1, 2, 3}         literal
a := [...]int{1, 2, 3}       let the compiler count
a := [5]int{2: 9}        [*] index into the literal: [0 0 9 0 0]
len(a)                       always the compile-time length
b := a                       COPIES the whole array
a == b                       arrays are comparable if the element type is
(you rarely want an array — use a slice)
```

SLICES: builtins

```
var s []int                  nil slice: len 0, cap 0, ready for append
s := []int{}                 empty but non-nil (differs only in JSON/==nil)
s := []int{1, 2, 3}          literal
make([]int, 3)               len 3, cap 3, zero-valued
make([]int, 0, 10)           len 0, cap 10 — preallocation
len(s) / cap(s)              length / capacity
append(s, v)                 ALWAYS reassign: s = append(s, v)
append(s, other...)          concatenate
copy(dst, src)               copies min(len) elements -> count
clear(s)                 [*] Go 1.21+: zero every element (keeps len)
s[i]                         index; panics if out of range
s[low:high]                  half-open: includes low, excludes high
s[:2] / s[2:] / s[:]         prefix / suffix / whole
s[low:high:max]          [*] three-index: also caps the new slice
```

SLICE INTERNALS (memorize)

```
a slice is 3 words           pointer to array, len, cap
passing a slice              copies the header, SHARES the backing array
s[1:3]                       a view: writes through to the same array
append within cap            writes in place — the original sees it
append past cap              allocates a new array, roughly doubling
big[:1] keeps big alive      re-slicing pins the whole backing array
dup := make([]T, len(s)); copy(dup, s)   the explicit deep-ish copy
slices.Clone(s)          [*] the one-liner version of the same thing
s == nil                     legal; s == other is NOT (use slices.Equal)
```

slices package (Go 1.21+) [*]

```
slices.Contains(s, v)        membership -> bool
slices.Index(s, v)           first index, or -1
slices.IndexFunc(s, f)       first index matching a predicate
slices.Sort(s)               ascending, for ordered types
slices.SortFunc(s, cmp)      custom comparison (return -1/0/+1)
slices.SortStableFunc(s, cmp)  keeps equal elements in order
slices.BinarySearch(s, v)    -> (index, found) on a sorted slice
slices.Reverse(s)            in place
slices.Equal(a, b)           element-wise comparison
slices.Max(s) / slices.Min(s)   panics on an empty slice
slices.Clone(s)              shallow copy
slices.Compact(s)            drop ADJACENT duplicates (sort first)
slices.Insert(s, i, v...)    insert at an index
slices.Delete(s, i, j)       remove a range
slices.Concat(a, b)          join slices (Go 1.22+)
slices.Collect(seq)          iterator -> slice (Go 1.23+)
slices.Sorted(seq)           iterator -> sorted slice (Go 1.23+)
```

MAPS

```
var m map[string]int         nil map: reads OK, WRITES PANIC
m := map[string]int{}        empty and usable
m := map[string]int{"a": 1}  literal
make(map[string]int)         same as the empty literal
make(map[string]int, 100) [*] preallocate for n keys
m[k] = v                     set (or overwrite)
v := m[k]                    read; missing key -> the zero value
v, ok := m[k]                the comma-ok form: ok tells you if it existed
delete(m, k)                 remove; deleting a missing key is a no-op
len(m)                       number of keys
clear(m)                 [*] Go 1.21+: remove every key
for k, v := range m          RANDOM order, every time, on purpose
(keys must be comparable: no slices, maps, or funcs as keys)
```

maps package (Go 1.21+) [*]

```
maps.Keys(m)                 iter.Seq of keys (use slices.Sorted to order them)
maps.Values(m)               iter.Seq of values
maps.Clone(m)                shallow copy
maps.Equal(a, b)             same keys and values
maps.DeleteFunc(m, f)        delete every entry matching a predicate
maps.Copy(dst, src)          merge src into dst
```

SETS (Go has no set type)

```
map[string]bool              readable; m[k] is false for absent keys
map[string]struct{}          zero bytes per entry — the memory-tight form
set[k] = struct{}{}          add
_, ok := set[k]              contains
delete(set, k)               remove
(iterate the map for union/intersection/difference — see the collections sheet)
```

STRINGS: the model

```
string                       IMMUTABLE bytes, conventionally UTF-8
len(s)                       BYTES, not characters
s[0]                         a byte (uint8) — not a character
s[0] = 'x'                   compile error: strings are immutable
for i, r := range s          decodes UTF-8: i is a byte index, r is a rune
[]rune(s)                    code points, so len() counts characters
[]byte(s)                    the raw bytes (a copy)
string(b) / string(r)        back from []byte / []rune
utf8.RuneCountInString(s) [*] character count without allocating
s + t                        concatenation (allocates every time)
`raw string`                 backticks: no escapes, newlines allowed
```

strings package

```
strings.Contains(s, sub)     substring test
strings.HasPrefix / HasSuffix   edge tests
strings.Index / LastIndex    byte position, or -1
strings.Count(s, sub)        non-overlapping occurrences
strings.Split(s, sep)        -> []string
strings.SplitN(s, sep, n) [*] at most n pieces
strings.Fields(s)            split on any run of whitespace
strings.Join(parts, sep)     -> string
strings.Replace(s, o, n, k)  k replacements (-1 = all)
strings.ReplaceAll(s, o, n)  every occurrence
strings.ToUpper / ToLower    case conversion
strings.TrimSpace(s)         strip leading/trailing whitespace
strings.Trim(s, cutset)      strip any of those characters
strings.TrimPrefix / TrimSuffix   remove one exact affix
strings.EqualFold(a, b)  [*] case-insensitive comparison
strings.Repeat(s, n)     [*] n copies
strings.Cut(s, sep)      [*] -> (before, after, found) — the modern split-in-2
strings.NewReader(s)     [*] an io.Reader over a string
strings.Builder              the efficient way to build a string
  b.WriteString(s) / b.WriteByte / b.WriteRune / b.Len / b.String / b.Grow
```

strconv

```
strconv.Itoa(i)              int -> string
strconv.Atoi(s)              string -> (int, error)
strconv.FormatInt(i, 10) [*] any base
strconv.ParseInt(s, 10, 64)  -> (int64, error)
strconv.ParseFloat(s, 64)    -> (float64, error)
strconv.ParseBool(s)     [*] "1","t","true","T","TRUE" ...
strconv.FormatFloat(f, 'f', 2, 64)  [*] precise float formatting
strconv.Quote(s)         [*] a Go-syntax quoted string
strconv.AppendInt(b, i, 10) [*] append without allocating
(strconv is the fast path; fmt.Sprintf is the flexible one)
```

unicode & bytes [*]

```
unicode.IsLetter / IsDigit / IsSpace / IsUpper / IsPunct   rune classes
unicode.ToUpper / ToLower    per-rune case
utf8.RuneCountInString(s)    character count
utf8.DecodeRuneInString(s)   -> (rune, size)
utf8.ValidString(s)          is it well-formed UTF-8?
bytes.Contains / Index / Split / Join / Trim...   the strings API for []byte
bytes.Buffer                 a growable byte buffer (io.Reader + io.Writer)
bytes.NewReader(b)           an io.Reader over a []byte
(work in []byte when the data comes from I/O — it avoids conversions)
```

fmt: printing

```
fmt.Println(a, b)            spaces between operands, newline at the end
fmt.Print(a, b)              spaces only between non-string operands
fmt.Printf(format, args...)  formatted, no automatic newline
fmt.Sprintf(...)             -> string
fmt.Fprintf(w, ...)          -> any io.Writer (os.Stderr, a file, a response)
fmt.Errorf("ctx: %w", err)   build a wrapped error
fmt.Sscanf / fmt.Sscan   [*] parse from a string
fmt.Fprintln(os.Stderr, ...) [*] the correct place for diagnostics
```

fmt: verbs

```
%v                           the default format
%+v                          structs WITH field names
%#v                          Go syntax — the debugging verb
%T                           the value's dynamic type
%d %b %o %x %X               integers in base 10/2/8/16
%f %.2f %e %g                floats: plain / fixed / exponent / compact
%s                           string, or anything with a String() method
%q                           double-quoted, escaped string
%c                           the character for a rune
%t                           boolean
%p                           pointer address
%w                           WRAP an error (fmt.Errorf only)
%%                           a literal percent sign
%8.2f / %-10s            [*] width, precision, left-align
```

TRAPS & MEMORIZE

```
writing to a nil map         panic — make it first
reading a nil map            fine, returns the zero value
append without reassigning   silently loses the result
s[1:3] shares memory         mutate the view, mutate the original
big[:1] holds the whole array  copy if you keep only a small piece
len(s) on a string           bytes, not characters
s[i] on a string             a byte; range gives you runes
string(65) vs strconv.Itoa   "A" vs "65" — go vet catches this one
map order is random          sort the keys before printing
+= in a loop to build strings  O(n²); use strings.Builder
comparing slices with ==     compile error; use slices.Equal
%v on an error inside Errorf  loses the chain — use %w
Printf without a newline     output looks interleaved and lost
```